

1. Write a C programs to implement multithreading where first thread calculates the sum of the elements of shared data (int data [10]), another thread finds the maximum value, and the third thread finds the minimum value. The main thread waits for these threads to finish and prints their results.

```
#include <iostream>
#include <thread>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

// Function to calculate the sum of elements
void calculateSum(const std::vector<int>& data, int& sum) {
    sum = 0;
    for (int num : data) {
        sum += num;
    }
}

// Function to find the maximum value
void findMax(const std::vector<int>& data, int& maxVal) {
    maxVal = INT_MIN;
    for (int num : data) {
        if (num > maxVal) {
            maxVal = num;
        }
    }
}

// Function to find the minimum value
void findMin(const std::vector<int>& data, int& minVal) {
    minVal = INT_MAX;
    for (int num : data) {
        if (num < minVal) {
            minVal = num;
        }
    }
}

int main() {
    // Shared data
    vector<int> data = {15, 22, 13, 27, 35, 19, 8, 24, 18, 30};
    int sum = 0, maxVal = 0, minVal = 0;

    // Create threads
    thread sumThread(calculateSum, std::cref(data), std::ref(sum));
    thread maxThread(findMax, std::cref(data), std::ref(maxVal));
    thread minThread(findMin, std::cref(data), std::ref(minVal));

    // Wait for all threads to finish
    sumThread.join();
    maxThread.join();
    minThread.join();
}
```

```
// Print results
cout << "Sum of elements: " << sum << std::endl;
cout << "Maximum value: " << maxVal << std::endl;
cout << "Minimum value: " << minVal << std::endl;

return 0;
}
```

OUTPUT:

```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab9$ ls
Lab9  Lab9.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab9$ cd ..
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files$ ls
Lab10  Lab2  Lab3  Lab4  Lab5  Lab6  Lab7  Lab8  Lab9
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files$ cd Lab10
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ls
Lab10_1.cpp  Lab10_2.cpp  Lab10_3.cpp  Lab10_4.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ g++ -o Lab10_1 Lab10_1.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ lsof | grep Lab10_1.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ g++ -o Lab10_1 Lab10_1.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ./Lab10_1
Sum of elements: 211
Maximum value: 35
Minimum value: 8
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$
```

2. Two threads thread1 and thread2 are updating the common variable inside a critical section. Write a program using semaphore to ensure that only one thread can access the critical section at a time, to prevent the race condition.

```
#include <iostream>
#include <thread>
#include <semaphore.h>
#include <unistd.h>

using namespace std;

// Global shared variable
int shared_variable = 0;

// POSIX semaphores to control access to the critical section
sem_t sem;

void updateSharedVariable(const std::string& threadName, int updateValue) {
    for (int i = 0; i < 5; ++i) {
        sem_wait(&sem); // Wait (P operation)

        // Critical section
        shared_variable += updateValue;
        cout << threadName << " updated shared variable to " << shared_variable <<
endl;

        sem_post(&sem); // Signal (V operation)

        sleep(1); // Simulate some work
    }
}

int main() {
    // Initialize the semaphore with a value of 1
    sem_init(&sem, 0, 1);

    // Create threads
    thread thread1(updateSharedVariable, "Thread 1", 1);
    thread thread2(updateSharedVariable, "Thread 2", -1);

    // Wait for threads to finish
    thread1.join();
    thread2.join();

    // Destroy the semaphore
    sem_destroy(&sem);

    cout << "Final value of shared variable: " << shared_variable << endl;

    return 0;
}
```

OUTPUT:

```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ls
Lab10_1  Lab10_1.cpp  Lab10_2.cpp  Lab10_3  Lab10_3.cpp  Lab10_4  Lab10_4.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ lsof | grep Lab10_2.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ g++ -o Lab10_2 Lab10_2.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ./Lab10_2
Thread 1 updated shared variable to 1
Thread 2 updated shared variable to 0
Thread 1 updated shared variable to 1
Thread 2 updated shared variable to 0
Thread 1 updated shared variable to 1
Thread 2 updated shared variable to 0
Thread 1 updated shared variable to 1
Thread 2 updated shared variable to 0
Thread 1 updated shared variable to 1
Thread 2 updated shared variable to 0
Thread 1 updated shared variable to 1
Thread 2 updated shared variable to 0
Final value of shared variable: 0
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$
```

3. Write a program in C to create a child process using fork() system call, parent and child shares a variable int VAR = 10; parent and child can execute concurrently, parent increments the value of VAR by 2 and child decrements the value by 2. Synchronize both the process by using semaphore such that child should always execute before parent.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <fcntl.h>

// Shared variable
int VAR = 10;

// Semaphore names
#define SEM_CHILD "/sem_child"
#define SEM_PARENT "/sem_parent"

int main() {
    pid_t pid;

    // Initialize semaphores
    sem_t *sem_child = sem_open(SEM_CHILD, O_CREAT, 0644, 1); // Initially allow the
child to run
    sem_t *sem_parent = sem_open(SEM_PARENT, O_CREAT, 0644, 0); // Initially block the
parent

    if (sem_child == SEM_FAILED || sem_parent == SEM_FAILED) {
        perror("sem_open");
        exit(EXIT_FAILURE);
    }

    pid = fork();

    if (pid < 0) {
        // Fork failed
        perror("fork");
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // Child process
        for (int i = 0; i < 5; ++i) {
            sem_wait(sem_child); // Wait for the semaphore to be available
            VAR -= 2; // Decrement VAR by 2
            printf("Child process: VAR = %d\n", VAR);
            sem_post(sem_parent); // Signal the parent process to run
        }
        exit(0);
    } else {
        // Parent process
        for (int i = 0; i < 5; ++i) {
            sem_wait(sem_parent); // Wait for the semaphore to be available
            VAR += 2; // Increment VAR by 2
```

```

        printf("Parent process: VAR = %d\n", VAR);
        sem_post(sem_child); // Signal the child process to run
    }

    // Wait for the child process to complete
    wait(NULL);

    // Cleanup semaphores
    sem_close(sem_child);
    sem_close(sem_parent);
    sem_unlink(SEM_CHILD);
    sem_unlink(SEM_PARENT);
}

return 0;
}

```

OUTPUT:

```

ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ lsof | grep Lab10_3.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ g++ -o Lab10_3 Lab10_3.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ./Lab10_3
Child process: VAR = 8
Parent process: VAR = 12
Child process: VAR = 6
Parent process: VAR = 14
Child process: VAR = 4
Parent process: VAR = 16
Child process: VAR = 2
Parent process: VAR = 18
Child process: VAR = 0
Parent process: VAR = 20
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$

```

4. Create a program that simulates a simple bank with multiple accounts and multiple clients making deposits (thread1) and withdrawals (thread2) concurrently. The goal is to ensure that account balances remain consistent even with concurrent operations. Use mutex locks to implement this.

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>

// Bank Account class
class BankAccount {
public:
    BankAccount(int id, double balance) : id(id), balance(balance) {}

    // Copy constructor
    BankAccount(const BankAccount& other) : id(other.id), balance(other.balance) {}

    // Deposit money into the account
    void deposit(double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        balance += amount;
        std::cout << "Deposited " << amount << " to account " << id << ". New balance: " << balance << std::endl;
    }

    // Withdraw money from the account
    void withdraw(double amount) {
        std::lock_guard<std::mutex> lock(mtx);
        if (balance >= amount) {
            balance -= amount;
            std::cout << "Withdrew " << amount << " from account " << id << ". New balance: " << balance << std::endl;
        } else {
            std::cout << "Insufficient funds in account " << id << std::endl;
        }
    }

    // Get the account balance
    double getBalance() const {
        std::lock_guard<std::mutex> lock(mtx);
        return balance;
    }

private:
    int id;
    double balance;
    mutable std::mutex mtx; // Mutex to protect the balance
};

// Function to simulate deposits
void depositMoney(BankAccount &account, double amount, int times) {
    for (int i = 0; i < times; ++i) {
```

```

        account.deposit(amount);
    }
}

// Function to simulate withdrawals
void withdrawMoney(BankAccount &account, double amount, int times) {
    for (int i = 0; i < times; ++i) {
        account.withdraw(amount);
    }
}

int main() {
    // Create bank accounts
    std::vector<BankAccount> accounts = {
        BankAccount(1, 1000.0),
        BankAccount(2, 2000.0),
        BankAccount(3, 3000.0)
    };

    // Create threads for deposits and withdrawals
    std::vector<std::thread> threads;

    // Simulate deposits
    for (auto &account : accounts) {
        threads.push_back(std::thread(depositMoney, std::ref(account), 100.0, 5));
    }

    // Simulate withdrawals
    for (auto &account : accounts) {
        threads.push_back(std::thread(withdrawMoney, std::ref(account), 50.0, 5));
    }

    // Join all threads
    for (auto &thread : threads) {
        thread.join();
    }

    // Print final balances
    for (const auto &account : accounts) {
        std::cout << "Final balance of account " << account.getBalance() << std::endl;
    }

    return 0;
}

```


OUTPUT:

```
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ls -l | grep Lab10_4.cpp
-rw-r--r-- 1 ubuntu ubuntu 1024 2023-10-10 10:10 Lab10_4.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ g++ -o Lab10_4 Lab10_4.cpp
ubuntu@ubuntu-VirtualBox:/media/sf_Ubuntu_Files/Lab10$ ./Lab10_4
Deposited 100 to account 2. New balance: 2100
Deposited 100 to account 2. New balance: 2200
Deposited 100 to account 2. New balance: 2300
Deposited 100 to account 2. New balance: 2400
Deposited 100 to account 2. New balance: 2500
100 to account 3. New balance: 3100
Deposited 100 to account 3. New balance: 3200
Deposited 100 to account 3. New balance: 3300
Deposited 100 to account 3. New balance: 3400
Deposited 100 to account 3. New balance: 3500
Withdrew 50 from account 3. New balance: 3450
Withdrew 50 from account 3. New balance: 3400
Withdrew 50 from account 3. New balance: 3350
Withdrew 50 from account 3. New balance: 3300
Withdrew 50 from account 3. New balance: 3250
Withdrew 50 from account 2. New balance: 2450
Withdrew 50 from account 2. New balance: 2400
Withdrew 50 from account 2. New balance: 2350
Withdrew 50 from account 2. New balance: 2300
Withdrew 50 from account 2. New balance: 2250
Deposited 100 to account 1. New balance: 1100
Deposited 100 to account 1. New balance: 1200
Withdrew 50 from account 1. New balance: 1150
Deposited 100 to account 1. New balance: 1250
Withdrew 50 from account 1. New balance: 1200
Deposited 100 to account 1. New balance: 1300
Withdrew 50 from account 1. New balance: 1250
Deposited 100 to account 1. New balance: 1350
Withdrew 50 from account 1. New balance: 1300
Withdrew 50 from account 1. New balance: 1250
Final balance of account 1250
Final balance of account 2250
Final balance of account 3250
```